PROJECT AND TEAM INFORMATION

Project Title

Memora: Saviours of memory

Team Name: *Team #*	*HydroLab* *OSDBMS-V-2026-T111*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	 *Kumar, Shivam – 24021338* *24021338@geu.ac.in*
Team member 2 *(Last Name, name: student ID: email, picture):*	 *Tarate, Sarthak– 240211326* *240211326@geu.ac.in*
Team member 3 *(Last Name, name: student ID: email, picture):*	 *Varun, Rudra – 240211920* *240211920@geu.ac.in*

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

MEMORA is a cost-gated, workload-transition-aware buffer management engine designed to optimize page replacement policies in database systems dynamically. Phase I established the core 5-module architecture and defined the PostgreSQL hook pointsbufmgr.c. In Phase II, we developed a C++/Python trace-driven simulator to evaluate sliding-window entropy transition detection and cost-benefit hysteresis gating against classical policies like LRU, LFU, and FIFO. Future work focuses on native PostgreSQL core integration and benchmarking under TPC-C/TPC-H workloads

Updated Project Approach and Architecture (2 pts)

Design: MEMORA employs a 5-component execution pipeline comprising a Workload Monitor, Transition Detector, Cost-Benefit Decision Gate, Policy Selector, and Buffer Manager. The C++ trace simulator handles frame allocations, while statistical drift calculations decide when to dynamically swap buffer policies.

Communication: The trace simulator logs trace accesses and statistics via structured JSON outputs and internal function callbacks, interfacing directly with policy selection modules.

Flow: The Workload Monitor captures page reference sequences. The Transition Detector monitors entropy variations ($\Delta S > \theta_T$). If a transition is detected, the Cost-Benefit Gate applies hysteresis ($B_{net} > \gamma$) to prevent policy thrashing before triggering a policy switch in the Buffer Manager.

Tools: Python, PostgreSQL (), C/C++ trace replay harness, Linux performance profiling utilities bufmgr.c.

Tasks Completed (7 pts)

Task Completed	Team Member
Core Buffer Pool and Eviction Engines	Sarthak Tarate
Workload Intelligence and Phase Detection	Shivam Kumar
Cost-Benefit Decision and Anti-Thrashing Engine	Shivam Kumar
Trace Simulator and Synthetic Generators	Rudra Varun
Testing and Quality Assurance:	Rudra Varun

Challenges/Roadblocks (7 pts)

Continuous Per-Access Overhead: Classical adaptive policies execute metadata updates on every page reference, wasting CPU cycles and memory bandwidth during stable workloads. Resolution: Implemented sliding-window statistical drift evaluation to restrict checks to window boundaries.

Blind Adaptation Without Cost-Benefit Awareness: Assuming that adapting always pays off leads to performance degradation when switching penalties outweigh benefits. Resolution: Integrated an analytical cost model balancing hit-rate improvements against migration and cold-miss penalties.

High Lock Contention in Concurrent DBMS: Complex multi-queue and ghost-list updates create locking bottlenecks on shared buffer frames. Resolution: Designed lightweight statistical sampling that bypasses heavy shared-list synchronization overheads.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Native PostgreSQL Core Integration: Hooking the transition detector and gating logic into PostgreSQL's shared buffer pool manager (src/backend/storage/buffer/bufmgr.c)	Sarthak Tarate
Live Workload Benchmarking: Replaying real-world database benchmarks using under TPC-C (OLTP) and TPC-H (OLAP) transactional load-spgbench.	Shivam Kumar
Real Trace Validation: Testing system performance using public production trace datasets such as the UMass Trace Repository and Storage Performance Council (SPC) traces.	Rudra Varun
Self-Tuning Hyperparameters: Applying lightweight reinforcement learning or online feedback heuristics to automatically tune window size W, detection threshold θ_T, and hysteresis parameter γ	Shivam Kumar

Project Outcome/Deliverables (2 pts)

A fully functional, modular MEMORA Python package with complete architectural specifications, developer guides, and algorithm documentation. A robust trace-driven simulation and benchmarking harness with swappable LRU, LFU, and FIFO eviction engines..

Verified test suite containing 60 out of 60 passing unit, integration, and regression tests. Phase acceptance reports, final research monograph detailing experimental findings, and publication-ready framework for academic venues.

Progress Overview (2 pts)

Done: Architecture design, core buffer pool management, statistical phase detector, cost-benefit anti-thrashing engine, trace simulator, synthetic workload generators, and comprehensive test suite validation are fully complete

Behind: Native PostgreSQL core modification () and live integration require final synchronization with concurrent database locks.

Not started: Live production trace validation (UMass/SPC) and online self-tuning hyperparameter optimization are scheduled for Phase III execution.

Codebase Information (2 pts)

Repository link: https://github.com/MEMORA-Engine/memora-core](https://github.com/MEMORA-Engine/memora-core)
Important commits: init: core architecture setup feat: sliding-window entropy detector test: 60/60 unit/integration/regression suite pass

Main folders: for core Python package containing,, and modules;for C++/python trace replay engine and synthetic workload generators; for unit, integration, and regression test suites; for PostgreSQL hook specifications and integration scripts; for architectural specs and research monograph. The files and are in the main folder.

Testing and Validation Status (2 pts)

Test Type	Status (Pass/Fail)	Notes
Unit tests	Pass	Validated frame operations, policy eviction invariants (LRU/LFU/FIFO), metric calculations, and cost gating threesholds (30 test cases).
Integration tests	Pass	Verified end-to-end adaptive pipeline execution across multi-phase workloads and PostgreSQL stats collector fallbacks (18 test cases).
Regresion Tests	Pass	Cross-referenced simullator outputs against hand-calculated reference trace baselines (12 test cases).
Trace Simulator Tests	Pass	Evaluated hit ratios, total I/O reads, policy switch counts, and post-shift recovery time on synthetic traces.
Native PostgreSQL Hook Tests	Pending	Direct integration tests inside PostgreSQL pending core patch compilation
Live TPC-C/TPC-H Workload Benchmarks	Pending	Full end-to-emd execution under mixed transactional/analytical loads scheduled for Phase III.

Deliverables Progress (2 pts)

Sliding-Window Phase Detector and Statistical Profiling Engine: **completed**
Trace Simulation Engine and Synthetic Workload Generators: **completed**
Comprehensive Test Suite (60/60 Passing): **completed**
Native PostgreSQL Core Integration: **in progress**
Real Workload Benchmarking (TPC-C / TPC-H): **pending**
Self-Tuning Hyperparameter Engine: **pending**